

Shoreline Sanctuary: Seagull Character & Sound Design Guide

This design guide outlines the official character style, sprite sheet layout, behavioral design, and custom audio assets for the **Seagull** character in the cozy beach game, *Shoreline Sanctuary*.

1. Character Concept & Design Philosophy

In keeping with the relaxing, low-stakes visual language of *Shoreline Sanctuary*, our Seagulls are designed to feel charming, plump, and friendly rather than loud or aggressive. They resemble rounded marshmallows with soft warm-white feathers, sandy-grey wings, and sweet apricot-yellow beaks and legs.

Core Visual Style Rules

- **Soft Outlines:** Use a warm, soft navy/slate outline instead of harsh black to evoke a gentle hand-drawn coloring-book or watercolor look.
- **Textured Finish:** Apply a subtle pencil/crayon overlay texture to the flat color fields.
- **Squash & Stretch:** Utilize a 5% squash on landings or waddles, and a 5% stretch on hard upward flap frames to add cartoon-style bounce.
- **Palette Callouts:**
 - **Feathers (Main Body):** Sand-White (HEX #F9F6F0)
 - **Wing Tips & Shadow:** Soft Slate Grey (HEX #8A95A5)
 - **Beak & Webbed Feet:** Apricot Yellow (HEX #F4B266)
 - **Sea Glass Trade Item:** Translucent Teal (HEX #46B29D)

2. Production-Ready Sprite Sheet Grid

The sprite sheet is configured on a **1024x1024 px canvas** split into an **8x8 grid** (each individual frame is **128x128 px**).

Row	Animation State	Frames	Description	Loopable?
Row 1	Idle (Standing)	1–6	Blinking, slight head tilts, and gentle tail wiggles while standing on the sand or rocks.	Yes
Row 2	Waddle (Walking)	1–8	Alternating webbed feet with	Yes

Row	Animation State	Frames	Description	Loopable?
			a clumsy, cute side-to-side hip sway.	
Row 3	Glide (Sailing)	1–6	Relaxed overhead flying with flat, static wings bobbing slightly on air currents.	Yes
Row 4	Swoop Down	1–6	Diving diagonally downward with wings tucked back securely. Climax of the swoop event.	No
Row 5	The Snatch / Trade	1–6	Reaching down with beak to grab picnic scraps or swap an item (e.g. dropping Sea Glass).	No
Row 6	Fly Away (With Loot)	1–8	Vigorous, heavier upward flapping while carrying an item safely in its beak.	Yes
Row 7	Floating on Water	1–6	Bobbing gently up and down on the ocean tides.	Yes
Row 8	Contented Sleep	1–4	Head nestled back into shoulders, chest slowly expanding and contracting to suggest breathing.	Yes

3. The Air-Drop & Trade Event Mechanic

Occasionally, as players clean up the beach, a seagull swoops down to trade items. This is designed to be a highly satisfying tactile event mapped directly to custom sound effects:

1. **The Swoop (Fwoosh):** The bird descends rapidly. A filtered windy noise rises dynamically.
2. **The Trade (Caw & Clack):** The seagull makes land or hovers, letting out a cute "Caw!" as the beak snaps down. A physical wooden/plastic "Clack" plays as it swaps a trash item for a piece of Sea Glass or a Soda Tab.
3. **The Departure:** The seagull pumps its wings and ascends back to the skies, letting out a fainter, second parting "Caw!"

4. Custom Python Audio Synthesis Script

The following script can be executed in any Python environment (requires numpy and scipy) to synthesize the uncompressed, CD-quality seagull_swoop.wav file from scratch.

```
import numpy as np
from scipy.io import wavfile

SAMPLE_RATE = 44100 # CD quality
DURATION = 2.0      # Total length of the sound effect
t = np.linspace(0, DURATION, int(SAMPLE_RATE * DURATION),
endpoint=False)

def create_fwoosh(t):
    """Generates a filtered noise sweep to simulate a seagull diving
    past."""
    noise = np.random.uniform(-1.0, 1.0, len(t))
    envelope = np.exp(-((t - 0.6) / 0.25) ** 2)
    window_size = 31
    smoothed_noise = np.convolve(noise,
np.ones(window_size)/window_size, mode='same')
    return smoothed_noise * envelope * 0.45

def create_caw(t):
    """Generates a double 'caw-caw' pitch-bent frequency
    modulation."""
    caw_signal = np.zeros_like(t)
    squawk_times = [(0.65, 0.25), (1.0, 0.22)]

    for start, duration in squawk_times:
        mask = (t >= start) & (t <= start + duration)
        if not np.any(mask):
            continue

        t_segment = t[mask] - start
        freq = 800 - 350 * (t_segment / duration)
```

```

        phase = 2 * np.pi * np.cumsum(freq) / SAMPLE_RATE
        tone = (np.sin(phase) +
                0.5 * np.sin(2 * phase) +
                0.25 * np.sin(3 * phase))

        envelope = np.sin(np.pi * (t_segment / duration))
        tremolo = 1.0 + 0.2 * np.sin(2 * np.pi * 35 * t_segment)
        caw_signal[mask] = tone * envelope * tremolo * 0.2

    return caw_signal

def create_clack(t):
    """Generates a tiny plastic/wooden 'clack' as the item is
    swapped."""
    clack_signal = np.zeros_like(t)
    start_time = 0.7
    duration = 0.05

    mask = (t >= start_time) & (t <= start_time + duration)
    t_segment = t[mask] - start_time
    freq = 1500
    tone = np.sin(2 * np.pi * freq * t_segment)
    decay_envelope = np.exp(-t_segment / 0.008)

    clack_signal[mask] = tone * decay_envelope * 0.25
    return clack_signal

fwoosh = create_fwoosh(t)
caw = create_caw(t)
clack = create_clack(t)
mixed_audio = fwoosh + caw + clack

max_val = np.max(np.abs(mixed_audio))
if max_val > 0:
    mixed_audio = mixed_audio / max_val * 0.8

audio_int16 = (mixed_audio * 32767).astype(np.int16)
wavfile.write("seagull_swoop.wav", SAMPLE_RATE, audio_int16)
print("Successfully generated 'seagull_swoop.wav'!")

```